Universidad del Valle de Guatemala

Facultad de Ingeniería

Proyecto 1: Detección de Ciberacoso en Redes Sociales

Curso: Inteligencia Artificial

Catedrático: Ingeniero Alan

Integrantes:

Genser Catalan

Jose Pablo

Hugo Barillas

Guatemala, 5 de abril de 2026

1. Introducción

En este proyecto desarrollamos un sistema de clasificación automática que identifica si un tweet contiene contenido de ciberacoso, y si lo tiene, en qué categoría cae, que puede ser acoso por edad, etnia, género, religión, otro tipo de ciberacoso, o simplemente que no es ciberacoso. Para lograrlo, aplicamos técnicas de procesamiento de lenguaje natural junto con modelos de clasificación supervisada que vimos durante el curso.

El conjunto de datos que usamos tiene 47,692 tweets etiquetados en seis categorías, y lo obtuvimos de Kaggle. Comparamos tres modelos distintos, Naive Bayes, Regresión Logística y una Red Neuronal Densa, para determinar cuál se desempeña mejor al momento de clasificar textos.

2. Descripción y Análisis de los Datos

2.1. Presentación del Dataset

El dataset que utilizamos se llama *Cyberbullying Classification* y contiene dos columnas: `tweet_text`, que tiene el texto del tweet, y `cyberbullying_type`, que indica la categoría a la que pertenece. En total hay 47,692 registros y no encontramos valores nulos en ninguna de las columnas, lo que nos indicó que el conjunto de datos estaba en buenas condiciones para trabajar.

2.2. Distribución de Clases

Una de las primeras cosas que revisamos fue si las clases estaban balanceadas, ya que un desbalance puede hacer que el modelo aprenda a favorecer una categoría sobre otras. Como se puede ver en la Figura 1, las seis categorías tienen una cantidad muy similar de tweets, con valores entre 7,823 y 7,998 por clase, lo que nos permitió trabajar sin necesidad de aplicar técnicas de balanceo como oversampling o undersampling.

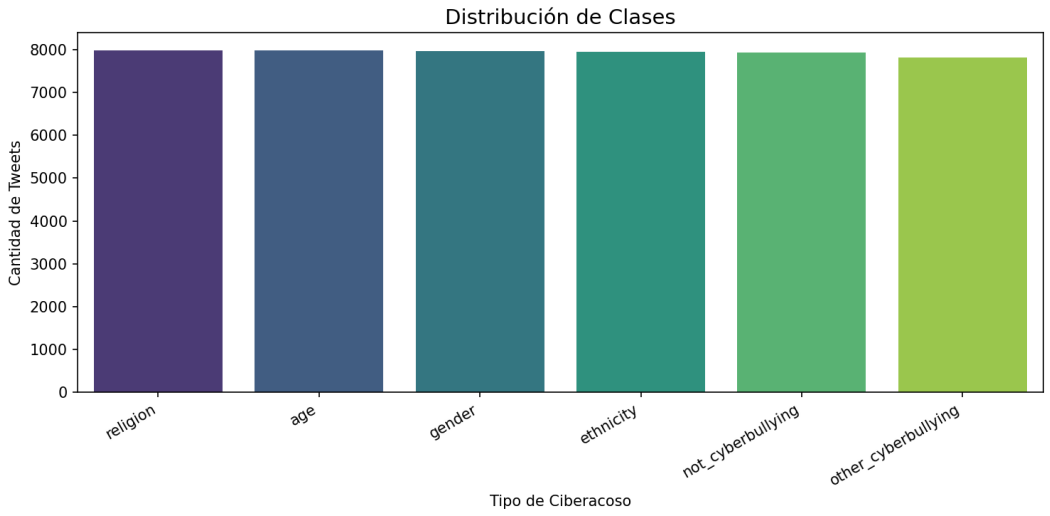

Figura 1: Distribución de clases en el dataset

2.3. Longitud de los Tweets

También analizamos la longitud de los tweets por categoría para ver si había diferencias notables. Como se muestra en la Figura 2, los tweets de la categoría *religion* tienen en promedio 198 caracteres, y los de *age* tienen 173, mientras que *not_cyberbullying* y *other_cyberbullying* tienen promedios mucho más cortos, de 83 y 85 caracteres respectivamente. También notamos algunos valores atípicos, que son tweets muy largos que se ven como puntos fuera de las cajas en el gráfico, pero decidimos no eliminarlos porque son tweets reales y válidos.

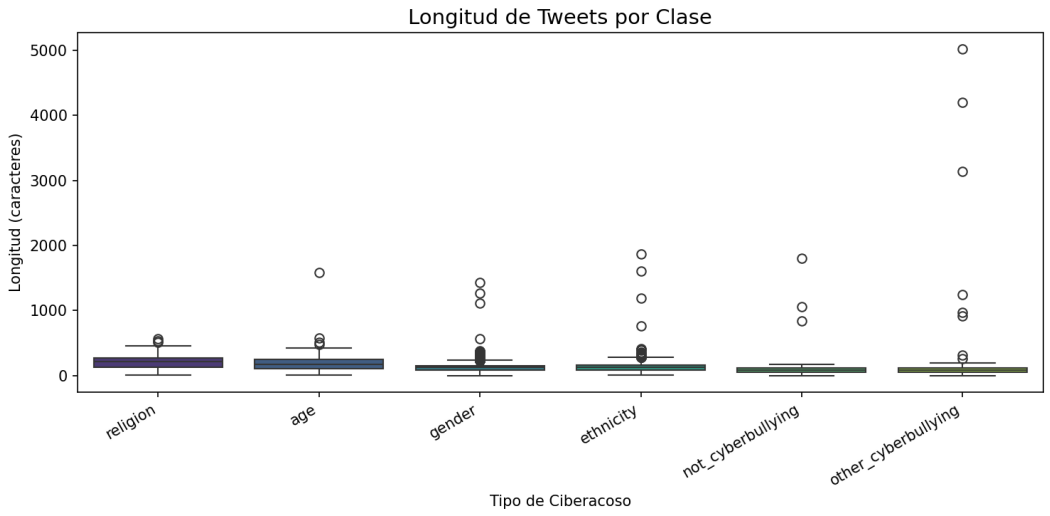

Figura 2: Longitud de tweets por categoría

La tabla a continuación resume las estadísticas de longitud por categoría:

Cuadro 1: Estadísticas de longitud de tweets por categoría

Categoría	**Cantidad**	**Media**	**Std**	**Mín**	**Máx**
religion	7,998	198.0	71.9	7	568
age	7,992	173.5	80.1	11	1,585
ethnicity	7,961	139.3	76.8	5	1,869
gender	7,973	136.4	71.4	2	1,431
other_cyberbullying	7,823	85.7	91.7	1	5,018
not_cyberbullying	7,945	83.1	45.5	2	1,809

3. Preprocesamiento de Texto

3.1. Pipeline de Limpieza

Antes de poder entrenar cualquier modelo, necesitamos convertir el texto en algo que los algoritmos puedan procesar. Para eso aplicamos una serie de pasos de limpieza a cada tweet en el siguiente orden: primero convertimos todo el texto a minúsculas para evitar que la misma palabra se cuente como dos por diferencia de mayúsculas, luego eliminamos las URLs, las menciones de usuarios que empiezan con arroba, y los hashtags, después quitamos todos los caracteres especiales, números y emojis, dejando solo letras del alfabeto, luego eliminamos las palabras vacías en inglés, que son palabras muy comunes que no aportan significado como *the*, *is* o *and*, y finalmente aplicamos lematización, que reduce las palabras a su forma base, por ejemplo *running* se convierte en *run*.

Un ejemplo del resultado de este proceso se puede ver a continuación:

- **Original:** *@XochitlSuckkks a classy whore? Or more red velvet cupcakes?*
- **Limpio:** *classy whore red velvet cupcake*

3.2. Datos Faltantes tras la Limpieza

Al terminar la limpieza, encontramos que 477 tweets quedaron completamente vacíos porque su contenido era únicamente hashtags o menciones, sin ninguna palabra de contenido real. Tomamos la decisión de eliminar esos registros, por lo que el dataset final con el que trabajamos quedó con 47,215 tweets.

3.3. Palabras más Frecuentes por Categoría

En la Figura 3 mostramos una nube de palabras que resume las palabras más frecuentes en cada categoría después de la limpieza. Se puede ver claramente cómo cada categoría tiene un vocabulario bastante distinto, lo que nos indicó que la tarea de clasificación debería ser posible.

Figura 3: Palabras más frecuentes por categoría

4. Vectorización y Partición del Dataset

4.1. División Train y Test

Para poder evaluar los modelos de manera justa dividimos el dataset en dos partes: un 80 % para entrenamiento con 37,772 tweets, y un 20 % para prueba con 9,443 tweets. Usamos el parámetro `stratify` para asegurarnos de que la distribución de clases quedara igual en ambas partes.

4.2. Vectorización con TF-IDF

Los modelos de machine learning no pueden leer texto directamente, así que necesitamos convertirlo a números. Para esto usamos TF-IDF, que significa Frecuencia de Término por Frecuencia Inversa de Documento, y lo que hace es asignarle un peso numérico a cada palabra según qué tan importante es en un tweet comparado con todos los demás tweets. Palabras que aparecen en casi

todos los tweets reciben un peso bajo porque no distinguen entre categorías, mientras que palabras que solo aparecen en ciertos tweets reciben un peso alto.

Configuramos el vectorizador para usar las 10,000 palabras más importantes y consideramos tanto palabras individuales como pares de palabras consecutivas, lo que en procesamiento de lenguaje se llama bigramas. Al final, cada tweet quedó representado como un vector de 10,000 números, y la matriz de entrenamiento tuvo dimensiones de 37,772 por 10,000.

Un punto importante es que ajustamos el vectorizador únicamente con los datos de entrenamiento y luego lo aplicamos al conjunto de prueba, para no contaminar la evaluación con información que el modelo no debería ver.

5. Modelos Probados

5.1. Modelo 1: Naive Bayes

Naive Bayes es un clasificador probabilístico que asume que todas las características, en este caso las palabras, son independientes entre sí. Aunque esta suposición rara vez se cumple en el lenguaje real, el modelo funciona bastante bien para clasificación de texto y es muy rápido de entrenar. Usamos la variante Multinomial, que es la más adecuada cuando los valores son conteos o frecuencias como en TF-IDF, con un hiperparámetro `alpha=0.1` que controla el suavizado para manejar palabras que no aparecen en el entrenamiento.

5.2. Modelo 2: Regresión Logística

La Regresión Logística es un modelo lineal que aprende a separar las clases buscando una frontera en el espacio de características. A diferencia de Naive Bayes, no asume independencia entre palabras y puede capturar relaciones más complejas entre las características. Lo configuramos con `C=1.0` que controla la regularización, un máximo de 1,000 iteraciones para asegurar que converja, y el optimizador `saga` que es eficiente para datasets grandes con múltiples clases.

5.3. Modelo 3: Red Neuronal Densa (MLP)

La Red Neuronal Densa o Perceptrón Multicapa es un modelo que aprende representaciones de los datos a través de capas de neuronas conectadas. Usamos una arquitectura con dos capas ocultas de 256 y 128 neuronas respectivamente, con función de activación ReLU, un tamaño de mini-batch de 256 muestras y una tasa de aprendizaje de 0.001. Entrenamos por 30 iteraciones y pudimos observar cómo el error fue disminuyendo consistentemente desde 0.828 en la primera iteración hasta 0.073 en la última.

6. Resultados y Comparación de Modelos

6.1. Tabla Comparativa

En la siguiente tabla resumimos los resultados de los tres modelos:

Cuadro 2: Comparación de métricas de los tres modelos

Modelo	Acc. Train	Acc. Test	F1 Test	Diferencia
Naive Bayes	84.30 %	76.97 %	0.7577	0.0733
Regresión Logística	88.34 %	81.79 %	0.8164	0.0656
Red Neuronal MLP	96.48 %	76.88 %	0.7682	0.1959

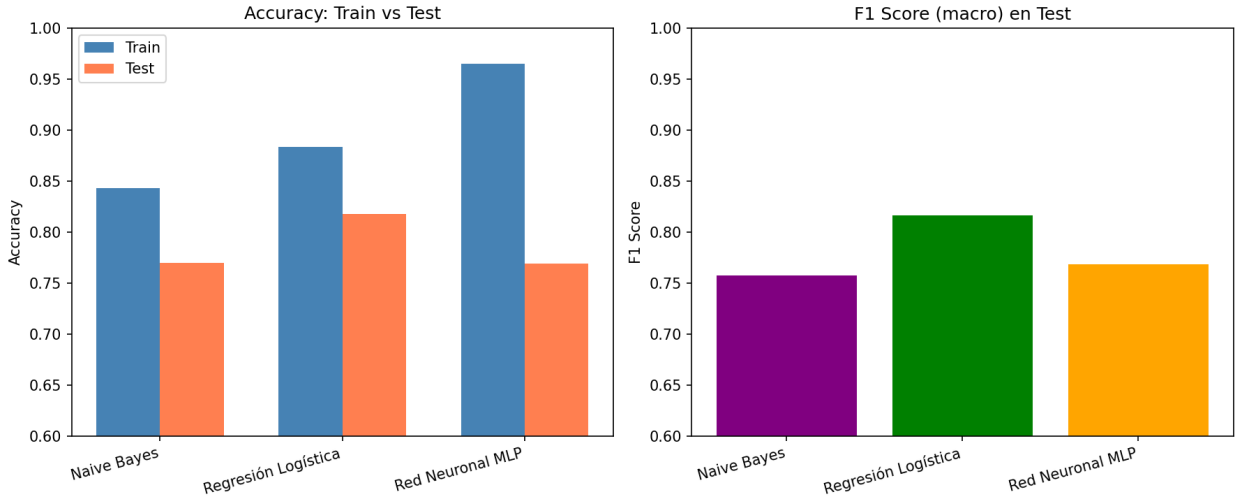

Figura 4: Comparación de accuracy y F1 Score entre los modelos

6.2. Matrices de Confusión

Las matrices de confusión nos permiten ver en detalle qué categorías se clasifican bien y cuáles se confunden entre sí.

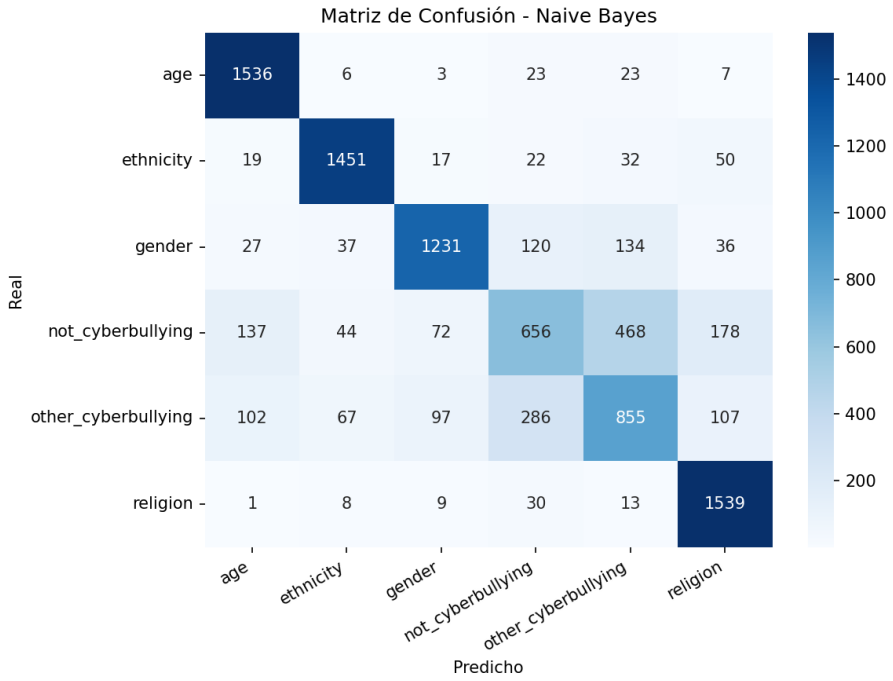

Figura 5: Matriz de confusión - Naive Bayes

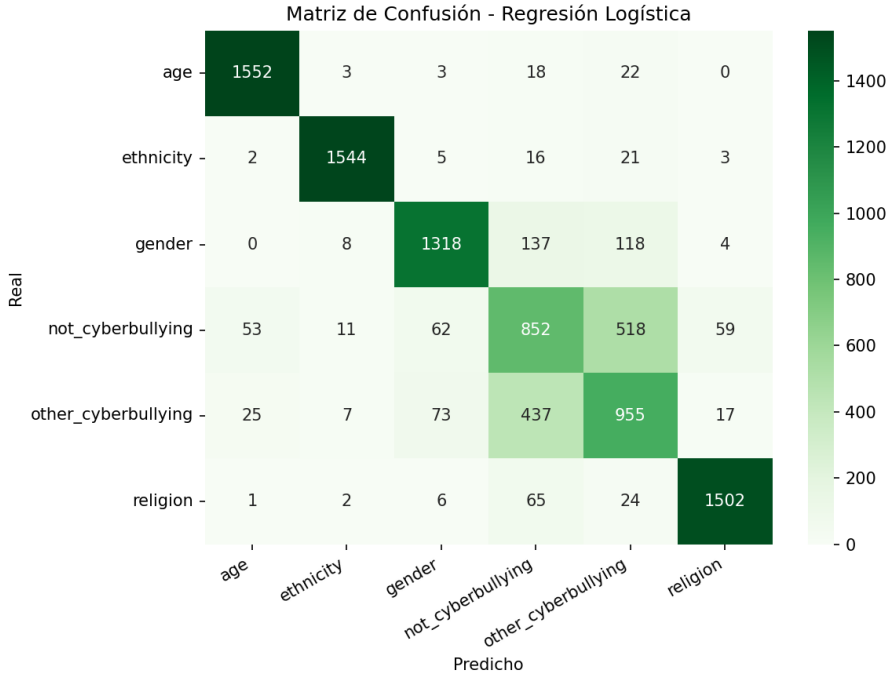

Figura 6: Matriz de confusión - Regresión Logística

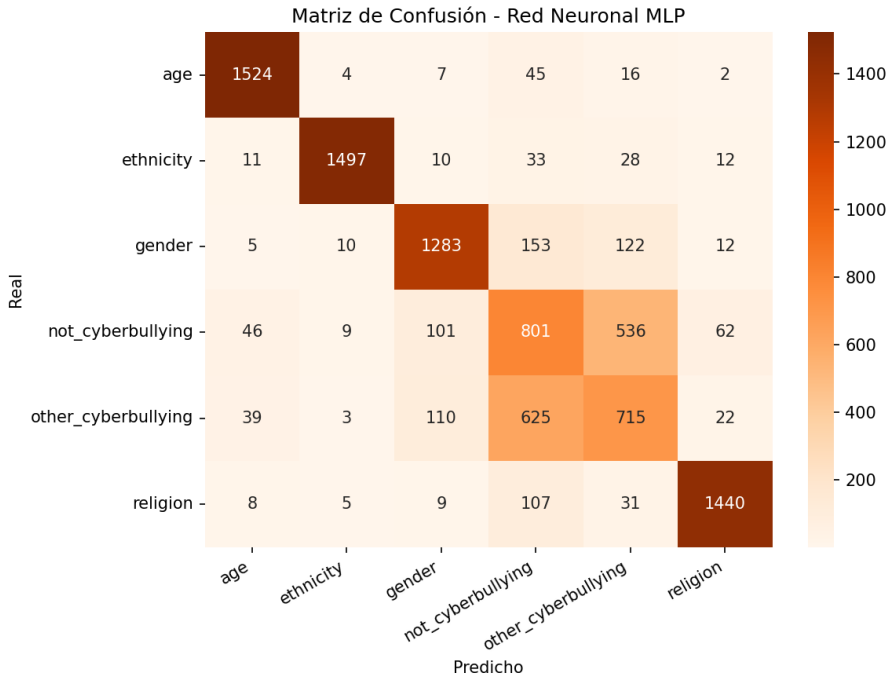

Figura 7: Matriz de confusión - Red Neuronal MLP

En los tres modelos se observa un patrón común: las categorías *religion*, *age* y *ethnicity* se clasifican con muy buena precisión, mientras que *not_cyberbullying* y *other_cyberbullying* se confunden entre sí con frecuencia, lo cual tiene sentido porque el límite entre un tweet que no es ciberacoso y uno que es ciberacoso genérico puede ser muy difuso incluso para una persona.

6.3. Ejemplos de Tweets Bien y Mal Clasificados

A continuación mostramos algunos ejemplos concretos del modelo de Regresión Logística:

Bien clasificados:

- *"Hence proved that a muslim won't accept that quran is Anti secular..."* → Clasificado correctamente como **religion**
- *"I'm not even exaggerating. All the girls who bullied me in high school..."* → Clasificado correctamente como **age**

Mal clasificados:

- *"This is 50 shades of bullshit. Wadhwa said that women that disagreed with him..."* → Real: **other_cyberbullying**, Predicho: **not_cyberbullying**

Este último error ilustra bien la dificultad de la tarea, ya que el tweet no contiene palabras claramente ofensivas pero sí describe una situación de acoso.

7. Justificación del Modelo Elegido

Elegimos la **Regresión Logística** como el mejor modelo por las siguientes razones:

Primero, obtuvo el mayor accuracy en el conjunto de prueba con un 81.79 %, superando a Naive Bayes por casi 5 puntos porcentuales y a la Red Neuronal por casi 5 puntos también, con el F1 Score más alto de 0.8164.

Segundo, y quizás más importante, es el modelo que menos sobreajuste presenta. La diferencia entre su accuracy de entrenamiento y de prueba es de solo 6.56 %, mientras que la Red Neuronal tiene una diferencia de 19.59 %, lo que indica que memorizó los datos de entrenamiento pero no aprendió a generalizar bien.

Tercero, la Regresión Logística es computacionalmente mucho más eficiente que la Red Neuronal y produce resultados mejores y más confiables en este tipo de tarea de clasificación de texto con TF-IDF.

8. Prueba de Usuario

Desarrollamos una función que recibe cualquier tweet como texto y devuelve la categoría predicha junto con las probabilidades de cada clase. A continuación mostramos tres ejemplos de predicciones:

Cuadro 3: Ejemplos de predicciones del sistema

Tweet	**Predicción**	**Confianza**
"You are too old to be doing this, grandpa"	age	79.13 %
"I love this sunny day, have a great weekend!"	not_cyberbullying	63.40 %
"People of that religion are all terrorists"	religion	96.31 %

Los resultados muestran que el modelo identifica correctamente los tres tipos de contenido, asignando alta confianza cuando el vocabulario es claro y confianza más moderada cuando el contenido es más ambiguo.

9. Conclusiones

A lo largo de este proyecto construimos un sistema completo de clasificación de ciberacoso, desde la carga y limpieza de los datos hasta la evaluación y comparación de tres modelos distintos. Aprendimos que el preprocesamiento del texto es una etapa crítica, ya que sin una buena limpieza los modelos reciben ruido innecesario que afecta su desempeño.

Comprobamos que la Regresión Logística es una herramienta muy poderosa para clasificación de texto, ya que combina buen desempeño con bajo riesgo de sobreajuste y un costo computacional razonable, superando incluso a la Red Neuronal en este caso, lo cual nos mostró que modelos más complejos no siempre significan mejores resultados.

También identificamos que las categorías más difíciles de clasificar son *not_cyberbullying* y *other_cyberbullying*, lo cual es comprensible porque el lenguaje de acoso genérico puede parecerse mucho al lenguaje cotidiano, y esa ambigüedad es un reto incluso para sistemas más sofisticados.